BLOCKSEC

Security Audit Report for Dstock Underlying

Date: December 22, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**

1.1 About Target Contracts . . . 1

1.2 Disclaimer . . . 1

1.3 Procedure of Auditing . . . 2

1.3.1 Security Issues . . . 2

1.3.2 Additional Recommendation . . . 3

1.4 Security Model . . . 3

Chapter 2 Findings **4**

2.1 Security Issue . . . 4

2.1.1 Unrestricted burn exposure via inheritance . . . 4

2.1.2 Lack of burning capability from blacklisted addresses . . . 5

2.2 Recommendation . . . 6

2.2.1 Add mutual exclusion logic for whitelist and blacklist . . . 6

2.2.2 Validate `compliance` contract configuration . . . 7

2.2.3 Validate implementation change in function `upgradeImplementation()` . . . 8

2.2.4 Declare `underlyingBeacon` as an immutable variable . . . 8

2.2.5 Use function `encodeCall()` to encode call data . . . 9

2.3 Note . . . 9

2.3.1 Integration risks regarding mutable token metadata . . . 9

2.3.2 Handling stock forward splits and reverse splits . . . 10

2.3.3 Potential centralization risks . . . 10

Report Manifest

Item	Description
Client	DStock
Target	Dstock Underlying

Version History

Version	Date	Description
1.0	December 22, 2025	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2025-12-23 03:54:41+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Solidity
Approach	Semi-automatic and manual verification

The target of this audit is the code repository [1] of Dstock Underlying of DStock.

The project is an upgradeable ERC20 token system through a factory pattern with Beacon Proxy architecture, including a token contract `DStockUnderlying`, a compliance contract `DStockUnderlyingCompliance`, and a factory contract `DStockUnderlyingFactory`. Specifically, `DStockUnderlying` incorporates role-based access control, requiring compliance checks for all transfers and allowing transactions to be paused. Token metadata can be reconfigured by the configurer role. Additionally, `DStockUnderlyingCompliance` integrates whitelist and blacklist mechanisms with two modes: default mode permitting all except blacklisted addresses, and whitelist-required mode allowing only whitelisted addresses. Furthermore, the contract `DStockUnderlyingFactory` enables batch creation of token instances with a shared upgradeable implementation and allows centralized upgrades.

Note this audit only focuses on the smart contracts in the following directories/files:

- src

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
dstock-underlying	`Version 1`	`e4756fdfba11deb0c5b4a5cbe4c972bf98886c52`
	`Version 2`	`95c4c17fd0d02e51412684f4c119af7de581bbf7`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset.

[1] https://github.com/dstockofficial/dstock-underlying

Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any warranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism

* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization
* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **two** potential security issues. Besides, we have **five** recommendations and **three** notes.

- Medium Risk: 2
- Recommendation: 5
- Note: 3

ID	Severity	Description	Category	Status
1	Medium	Unrestricted burn exposure via inheritance	Security Issue	Fixed
2	Medium	Lack of burning capability from blacklisted addresses	Security Issue	Fixed
3	-	Add mutual exclusion logic for whitelist and blacklist	Recommendation	Fixed
4	-	Validate `compliance` contract configuration	Recommendation	Fixed
5	-	Validate implementation change in function `upgradeImplementation()`	Recommendation	Fixed
6	-	Declare `underlyingBeacon` as an immutable variable	Recommendation	Fixed
7	-	Use function `encodeCall()` to encode call data	Recommendation	Fixed
8	-	Integration risks regarding mutable token metadata	Note	-
9	-	Handling stock forward splits and reverse splits	Note	-
10	-	Potential centralization risks	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Unrestricted burn exposure via inheritance

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In contract `DStockUnderlying`, function `burn()` is customized and protected by the modifier `onlyRole(BURNER_ROLE)`. However, the contract inherits from OpenZeppelin's contract `ERC20BurnableUpgradeable`. The inherited public functions `burn()` and `burnFrom()` are not overridden to include the same access control and remain publicly accessible. Consequently, any token holder can directly invoke the function `burn()` to destroy their own tokens or `burnFrom()` to destroy tokens from an approved allowance.

```
function burn(address from, uint256 amount)
    external
```

```
        onlyRole(BURNER_ROLE)
    {
        _burn(from, amount);
    }
```

Listing 2.1: src/DStockUnderlying.sol

```
contract DStockUnderlying is
    Initializable,
    ERC20BurnableUpgradeable,
    PausableUpgradeable,
    AccessControlUpgradeable,
    IDStockUnderlyingToken
{
```

Listing 2.2: src/DStockUnderlying.sol

```
    function burn(uint256 value) public virtual {
        _burn(_msgSender(), value);
    }

    /**
     * @dev Destroys a ‘value‘ amount of tokens from ‘account‘, deducting from
     * the caller’s allowance.
     *
     * See {ERC20-_burn} and {ERC20-allowance}.
     *
     * Requirements:
     *
     * - the caller must have allowance for ‘‘accounts‘‘’s tokens of at least
     * ‘value‘.
     */
    function burnFrom(address account, uint256 value) public virtual {
        _spendAllowance(account, _msgSender(), value);
        _burn(account, value);
    }
```

Listing 2.3: openzeppelin-contracts-upgradeable/contracts/token/ERC20/extensions/ERC20BurnableUpgradeable.sol

Impact This allows any token holder to burn tokens arbitrarily, circumventing intended privileged restrictions and potentially disrupting token economics.

Suggestion Override the inherited functions `burn()` and `burnFrom()` to apply the modifier `onlyRole(BURNER_ROLE)`, or disable the inherited burn functionality entirely.

2.1.2 Lack of burning capability from blacklisted addresses

Severity Medium

Status Fixed in `Version 2`

Introduced by `Version 1`

Description When `BURNER_ROLE` invokes function `burn()`, the function `_checkCompliance()` is triggered to check the address `from`'s compliance. If the address `from` is in the blacklist, the function reverts and forbids tokens held by blacklisted addresses from being burned.

```
function burn(address from, uint256 amount)
    external
    onlyRole(BURNER_ROLE)
{
    _burn(from, amount);
}
```

Listing 2.4: src/DStockUnderlying.sol

```
function _update(
    address from,
    address to,
    uint256 value
) internal virtual override {
    if (paused()) {
        revert TokenPaused();
    }

    if (from != address(0)) {
        _checkCompliance(from);
    }
```

Listing 2.5: src/DStockUnderlying.sol

Impact The protocol cannot burn tokens from blacklisted addresses, even if funds need to be burned in accordance with legal or compliance department requirements.

Suggestion Modify the check to allow skipping compliance checks when burning from a blacklisted address.

2.2 Recommendation

2.2.1 Add mutual exclusion logic for whitelist and blacklist

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In contract `DStockUnderlyingCompliance`, functions `setWhitelist()` and `setBlacklist()` manage separate lists without enforcing mutual exclusivity. A user could be added to both the whitelist and blacklist simultaneously. The compliance check logic in function `checkIsCompliant()` prioritizes the blacklist, but allowing dual status creates unnecessary complexity and potential confusion regarding the intended access control state.

```
/// @notice Batch update whitelist entries
function setWhitelist(address[] calldata users, bool allowed) external onlyOwner {
    uint256 len = users.length;
    for (uint256 i = 0; i < len; ) {
        whitelisted[users[i]] = allowed;
```

```
        emit WhitelistUpdated(users[i], allowed);

        unchecked {
            ++i;
        }
    }
}

/// @notice Batch update blacklist entries
function setBlacklist(address[] calldata users, bool blocked) external onlyOwner {
    uint256 len = users.length;
    for (uint256 i = 0; i < len; ) {
        blacklisted[users[i]] = blocked;
        emit BlacklistUpdated(users[i], blocked);

        unchecked {
            ++i;
        }
    }
}
```

Listing 2.6: src/DStockUnderlyingCompliance.sol

Suggestion Modify both functions `setWhitelist()` and `setBlacklist()` to automatically clear the opposite list status when setting a new one.

2.2.2 Validate `compliance` contract configuration

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockUnderlying`, the function `setCompliance()` lacks a validation to ensure the new compliance contract is correctly bound to this token. If the compliance contract's `underlyingToken` is not configured as `address(this)`, all subsequent compliance-dependent operations will revert.

```
/// @notice Set or update the compliance contract
function setCompliance(address newCompliance) external onlyRole(CONFIGURER_ROLE) {
    if (newCompliance == address(compliance)) {
        revert ValueUnchanged();
    }
    emit ComplianceChanged(address(compliance), newCompliance);
    compliance = IDStockUnderlyingCompliance(newCompliance);
}
```

Listing 2.7: src/DStockUnderlying.sol

```
contract DStockUnderlyingCompliance is IDStockUnderlyingCompliance, Ownable {
    /// @notice Underlying asset token (usually the DStockUnderlying proxy)
    address public immutable underlyingToken;
```

Listing 2.8: src/DStockUnderlyingCompliance.sol

Suggestion Add a validation to verify that the compliance contract's `underlyingToken` matches the current contract address before assignment.

2.2.3 Validate implementation change in function `upgradeImplementation()`

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In the contract `DStockUnderlyingFactory`, the function `upgradeImplementation()` allows upgrading to the same implementation address currently in use. This results in a redundant state change and emits a misleading event `ImplementationUpgraded`.

```
/**
 * @notice Upgrade the DStockUnderlying implementation (affects all proxies)
 * @param newImplementation Address of the new implementation
 */
function upgradeImplementation(address newImplementation)
    external
    onlyRole(UPGRADER_ROLE)
{
    if (newImplementation == address(0)) revert InvalidImplementation();
    address oldImpl = underlyingBeacon.implementation();
    underlyingBeacon.upgradeTo(newImplementation);
    emit ImplementationUpgraded(oldImpl, newImplementation);
}
```

Listing 2.9: src/DStockUnderlyingFactory.sol

Suggestion Verify that the `newImplementation` differs from the current beacon implementation before proceeding with the upgrade.

2.2.4 Declare `underlyingBeacon` as an immutable variable

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In contract `DStockUnderlyingFactory`, variable `underlyingBeacon` is set during construction. The variable is used as the shared Beacon across underlying contracts, and cannot be modified. Therefore, it is recommended to declare the variable as an immutable variable, which saves gas costs during variable reads.

```
/// @notice Beacon shared by all DStockUnderlying proxies
UpgradeableBeacon public underlyingBeacon;
```

Listing 2.10: src/DStockUnderlyingFactory.sol

```
constructor(address impl, address admin) {
    if (impl == address(0)) revert InvalidImplementation();
    if (admin == address(0)) revert ZeroAddress();

    _grantRole(DEFAULT_ADMIN_ROLE, admin);
    _grantRole(DEPLOYER_ROLE, admin);
```

```
        _grantRole(UPGRADER_ROLE, admin);

        // Factory is the beacon owner to enable upgradeTo calls
        underlyingBeacon = new UpgradeableBeacon(impl, address(this));
    }
```

Listing 2.11: src/DStockUnderlyingFactory.sol

Suggestion Declare variable `underlyingBeacon` as immutable.

2.2.5 Use function `encodeCall()` to encode call data

Status Fixed in `Version 2`

Introduced by `Version 1`

Description In function `createUnderlying()`, it is recommended to use the function `encodeCall()` to encode the call data for initialization instead of `encodeWithSelector()`. Function `encodeCall()` performs type checks to ensure inputs actually match the types expected by the invoked function.

```
    function createUnderlying(
        string memory name_,
        string memory symbol_,
        uint8 decimals_,
        address admin,
        address compliance
    ) external onlyRole(DEPLOYER_ROLE) returns (address) {
        if (admin == address(0)) revert ZeroAddress();
        bytes memory data = abi.encodeWithSelector(
            IDStockUnderlyingToken.initialize.selector,
            name_,
            symbol_,
            decimals_,
            admin,
            compliance
        );
```

Listing 2.12: src/DStockUnderlyingFactory.sol

Suggestion Replace function `encodeWithSelector()` with function `encodeCall()`.

2.3 Note

2.3.1 Integration risks regarding mutable token metadata

Introduced by `Version 1`

Description Contract `DStockUnderlying` allows privileged roles to update fundamental token metadata, including `name` and `symbol`. These modifications may occur in response to changes in the underlying stock. While this provides administrative flexibility, it introduces potential risks for third-party integrations. Many protocols store token metadata during initialization and use

the stored values for internal accounting. If the underlying token's metadata changes post-integration, protocols using cached values may experience data inconsistencies. To mitigate such risks, projects integrating with this contract must account for mutable metadata by implementing appropriate solutions.

2.3.2 Handling stock forward splits and reverse splits

Introduced by `Version 2`

Description Contract `DStockUnderlying` is pegged to a real-world stock asset. Traditional equity markets occasionally conduct corporate actions such as stock splits or reverse stock splits. To mirror a ten-for-one forward stock split, the project would manually airdrop (i.e., mint) additional tokens to all existing holders. Conversely, a reverse split would require a corresponding destruction (i.e., burn) of tokens. This process involves significant gas costs for state updates across all holder addresses.

2.3.3 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., `DEFAULT_ADMIN_ROLE`, `owner`) can conduct sensitive operations, which introduces potential centralization risks. For example, the owner of contract `DStockUnderlyingCompliance` holds unilateral authority over critical access controls of token operations, such as toggling the whitelist requirement and arbitrarily adding or removing addresses from both the `whitelist` and `blacklist`. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS